

BÁO CÁO BÀI TẬP LỚN: HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN vBay

HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN THỜI GIAN THỰC (vBay)

- **Môn học:** Lập trình nâng cao
- **Dự án:** vBay (Desktop App Client-Server Realtime)
- **Repository:** <https://github.com/cuongdz2k7/vbay>
- **Kiến trúc:** Multi-module Maven

1. MỤC TIÊU VÀ PHẠM VI THỰC HIỆN

1.1. Mục tiêu dự án

Dự án **vBay** xây dựng sàn đấu giá Desktop App theo mô hình Client-Server realtime. Bài toán chính không chỉ là cho người dùng đặt giá, mà còn phải bảo đảm nhiều người có thể thao tác cùng lúc mà dữ liệu giá, người thắng và số dư ví vẫn nhất quán. Vì vậy hệ thống tập trung vào **OOP/MVC/Design Patterns**, xử lý **concurrency**, truyền sự kiện qua **TCP Socket** và mô phỏng cơ chế ví kỹ quỹ an toàn.

Về nghiệp vụ, vBay **không tách riêng tài khoản seller và bidder**: một tài khoản **USER** có thể vừa đăng bán sản phẩm/tạo phiên đấu giá, vừa tham gia đặt giá ở các phiên khác. Người dùng có thể **bid thủ công**, đăng ký **auto-bid**, mua ngay bằng **Buy Now** và theo dõi diễn biến giá **realtime**. **ADMIN** giữ vai trò quản trị hệ thống như duyệt ví, khóa/cảnh cáo/kick user và xử lý các trường hợp vi phạm. Các tính năng **Anti-sniping**, **reserve price**, **BidPriceChart** và **Self-Healing Scheduler** giúp phiên đấu công bằng hơn, dễ quan sát hơn và ổn định hơn khi server gặp sự cố.

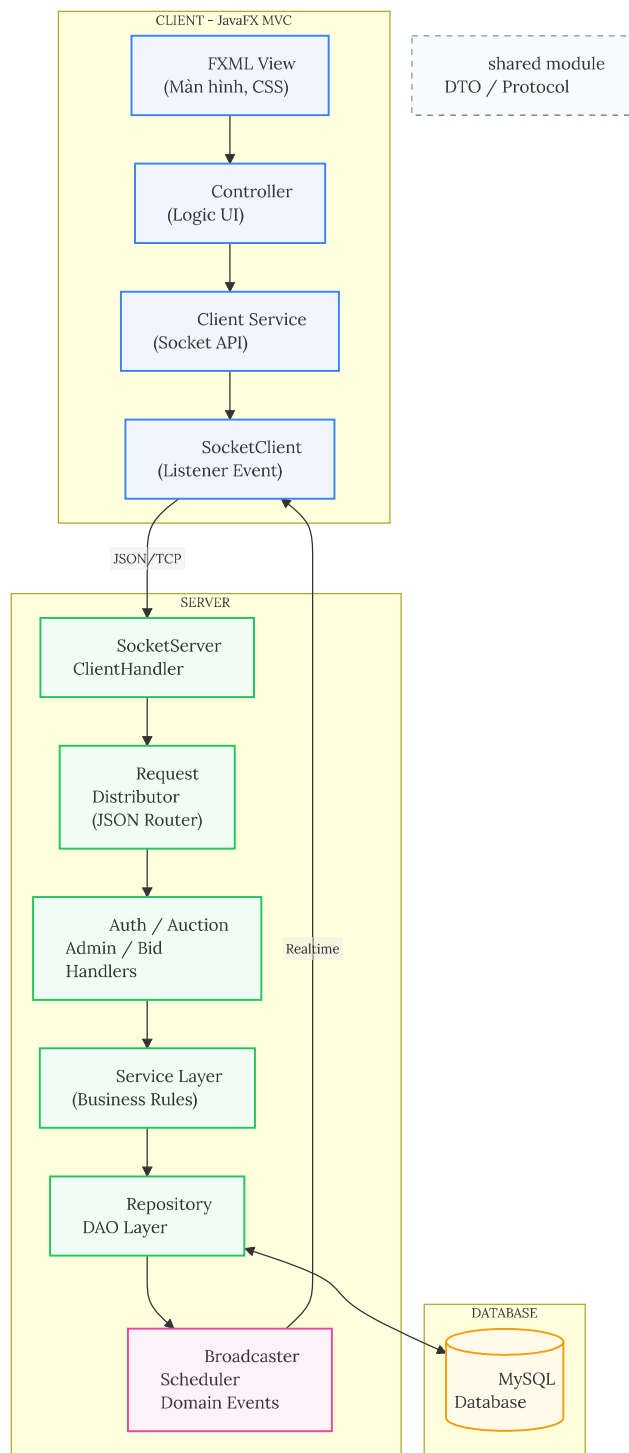
1.2. Phạm vi thực hiện

- **Môi trường:** JavaFX client, TCP server đa luồng, phân quyền **USER** và **ADMIN**; **USER** có thể đồng thời là người bán và người đấu giá.
- **Lưu trữ & test:** MySQL, H2 in-memory, JUnit 5/Mockito, GitHub Actions; không bao gồm cloud, thanh toán thật, SSL/TLS.
- **Luồng chính:** đăng ký/dăng nhập, tạo sản phẩm, tạo phiên đấu giá, **đặt bid**, **auto-bid**, **mua ngay**, theo dõi **realtime**, quản trị user và xử lý kết thúc phiên.

2. KIẾN TRÚC TỔNG THỂ CỦA HỆ THỐNG

2.1. Sơ đồ kiến trúc tổng thể

Hệ thống gồm 3 module Maven: **shared** (DTO/protocol), **client** (JavaFX MVC), **server** (service/repository/database). **shared** giữ hợp đồng giao tiếp để client và server không lệch model; **client** chỉ xử lý giao diện và gửi request; **server** chịu trách nhiệm **xác thực**, **kiểm tra quyền**, **xử lý nghiệp vụ** và **ghi database**. Cách chia này giúp phần **đấu giá**, **ví tiền** và **realtime** được kiểm soát tập trung ở server thay vì phân tán trong UI.



2.2. Mô tả chi tiết theo sơ đồ kiến trúc

- **Module & MVC:** `shared` chứa **DTO/protocol** dùng chung; `client` tách **FXML/CSS/controller**; `server` chia **Distributor -> Service -> DAO** để controller không chứa nghiệp vụ.
- **Realtime:** **TCP port 3618** dùng **Line-Delimited JSON** để tránh dính gói tin; `SubscriptionRegistry` broadcast theo room `AUCTION_LIST`, `AUCTION_id`, `USER_id`.
- **Bidding pipeline:** `AuctionBidEngine` quyết định **bid/winner/reserve**, `BidResolutionApplier` ghi DB trong **transaction** có `FOR UPDATE`; scheduler quét lại DB khi

khởi động hoặc theo chu kỳ để lập lịch lại các phiên còn hiệu lực sau crash.

3. CHỨC NĂNG ĐẠT ĐƯỢC THEO BAREM ĐIỂM

Nhóm chức năng	Hướng giải quyết	Lý do
Thiết kế OOP (2.5đ)	• Lớp chính: User, Product, ProductImage, Auction, Bid, Autobid, Payment, DTO trong shared. • Encapsulation: field private, getter/setter; dùng protected cho phần cần lớp con truy cập như AbstractRealtimeHandler và constructor BidUpdateResult; quyền sở hữu lấy từ ClientSession. • Inheritance: AppException, AbstractRealtimeHandler, BidUpdateResult. • Polymorphism/abstraction: AuctionChange, BidFinalization, DomainEventHandler, repository interface.	• Seller/bidder là ngữ cảnh của cùng USER, xác định qua auctions.seller_id và bids.bidder_id. • Interface giúp service/applier gọi theo hợp đồng chung, dễ mở rộng.
Danh mục (2.5đ)	Mỗi Product chỉ lưu categoryId để trỏ tới bảng danh mục trong DB . Bảng danh mục có thể lưu quan hệ cha-con , ví dụ Điện tử -> Điện thoại -> Smartphone.	Category là dữ liệu thay đổi thường xuyên nên đặt trong DB thay vì tạo nhiều subclass Java. Cách này cho phép thêm/sửa danh mục mà không cần đổi code hoặc build lại ứng dụng.
Design Pattern	• MVC: JavaFX FXML + Controller + model. • DAO/Repository: JDBC repositories. • Dependency Injection: qua AppConfig. • Observer/Event: DomainEventPublisher + RealtimeBroadcaster. • Builder: cho BidResolution và RoomFilter/room key realtime. • Singleton: cho SocketClient/registries.	• Tách UI/logic và cô lập truy cập DB. • Giảm phụ thuộc cứng, phù hợp realtime và quản lý tài nguyên dùng chung.
User/Product (3.0đ)	AuthService, Argon2 , session socket ; USER vừa có thể bán vừa có thể bid; admin ban/lock/kick/duyet ví . User tạo product/auction và upload ảnh qua HTTP image server port 1639 .	Tách user/product khỏi client giúp server kiểm soát quyền, giảm gian lận và dễ test business rule.
Đấu giá (3.0đ)	ManualBidService, AutobidService, BuyNowService điều phối; AuctionBidEngine tính bid hợp lệ, reserve price, winner, buy now ; BidResolutionApplier ghi DB.	Bidding có nhiều side effect tài chính nên dùng pipeline riêng để tách quyết định nghiệp vụ khỏi thao tác database.
Lỗi & ngoại lệ (3.0đ)	Dùng ValidationException, AuthenticationException, bắt lỗi ở RequestDistributor và trả Respond(success=false). Transaction rollback khi có SQLException hoặc runtime error.	Client nhận lỗi nhất quán; server không commit nửa vời khi bid/deposit/buy-now gặp lỗi.

Nhóm chức năng	Hướng giải quyết	Lý do
Đồng thời (1.5đ)	Mỗi client có ClientHandler riêng. Các flow bid/buy-now/auto-bid dùng setAutoCommit(false), lockAuctionForUpdate, lockUserForUpdate, commit/rollback rõ ràng; entity có version .	Khóa bi quan ở hàng auction/user ngăn lost update , double spending và race condition khi nhiều bidder đặt giá cùng lúc.
Realtime (1.5đ)	Client subscribe room ; server phát RealtimeEvent qua RealtimeBroadcaster. Event gồm bid history , auction state , balance , watcher count , autobid update .	Room-based broadcasting tránh gửi dữ liệu riêng tư sai người và giảm tải so với broadcast toàn hệ thống.
Client-server, MVC, DAO (2.0đ)	JavaFX/FXML controller gọi SocketClient; server nhận JSON qua TCP và phân phối tới service; repository interface + JDBC implementation truy cập MySQL .	Biên module rõ, dễ thay DB/UI/protocol hơn so với viết logic trực tiếp trong controller.
Maven, clean code (2.0đ)	Project chia module client/server/shared, dùng Maven dependency/plugin , package fat JAR bằng assembly plugin; có tài liệu kỹ thuật trong report/.	Build và phát hành nhất quán ; shared DTO tránh lặp model giao tiếp giữa client và server.
Unit test & CI/CD (2.0đ)	Có JUnit 5/Mockito , AntiSnipePolicyTest, AuctionServiceTest, AuctionServiceIntegrationTest, AuthServiceTest. GitHub Actions build/test và attach JAR release.	Kiểm tra các logic rủi ro cao như anti-snipe , auth , auction và giảm lỗi khi merge nhiều branch.
Nâng cao (1.5đ)	Auto-bidding , anti-sniping , Buy Now , reserve price , BidPriceChart, escrow wallet , self-healing scheduler .	Các tính năng này làm phiên đấu công bằng hơn , trực quan hơn và an toàn hơn về trạng thái ví tiền .

4. KỸ THUẬT NÂNG CAO VÀ ĐIỂM NỔI BẬT

4.1. Tự thầu (Auto-bidding)

- Người dùng đặt `maxBidAmount`, hệ thống chỉ tăng giá công khai ở mức vừa đủ để giữ vị trí dẫn đầu. Hạn mức tối đa được **giữ riêng tư** và chỉ gửi qua response/event cá nhân.

4.2. Chống bắn tỉa (Anti-sniping)

- `AntiSnipePolicy` phát hiện bid hợp lệ sát giờ kết thúc và sinh `AntiSnipeAuctionExtensionChange` để gia hạn `ending_time`. Nhờ vậy bidder khác vẫn có thời gian phản hồi, tránh thắng thầu bằng cú đặt giá phút cuối.

4.3. Biểu đồ biến động giá (Bid history visualization)

- Client nhận `BID_HISTORY_ITEM_ADDED` và cập nhật trực tiếp `BidPriceChart`, không cần reload danh sách bid. Biểu đồ giúp quan sát tốc độ tăng giá và các thời điểm cạnh tranh mạnh trong phiên.

4.4. Buy Now và Reserve Price

- Buy Now** cho phép chốt đơn ngay theo `buy_now_price`, kết thúc phiên và tạo payment `HELD`. `reserve_price` là giá sàn bảo vệ seller; nếu kết thúc chưa đạt reserve thì auction

FAILED và hold được release.

4.5. Động cơ ký quỹ ví đóng băng (Wallet hold escrow engine)

- Ví tách **available_balance** và **hold_balance** để mô phỏng **escrow**. Khi winner thay đổi, hệ thống **hold tiền winner mới** và **release người bị outbid** trong cùng transaction, tránh một khoản tiền được dùng cho nhiều phiên.

4.6. Bộ lập lịch tự phục hồi (Self-Healing Scheduler)

- **Scheduler** chịu trách nhiệm mở/kết thúc phiên đấu giá đúng thời điểm. Khi server **restart/crash**, hệ thống không phụ thuộc hoàn toàn vào lịch cũ trong bộ nhớ mà **quét lại DB** để tìm các auction còn hiệu lực, auction đã quá hạn nhưng chưa đóng và các phiên cần cập nhật trạng thái. Nhờ vậy lịch đấu giá được **khôi phục từ dữ liệu bền vững**, hạn chế phiên bị bỏ quên hoặc kẹt trạng thái sau sự cố.